

Prompting Large Language Models

Vinay Setty

vinay.j.setty@uis.no

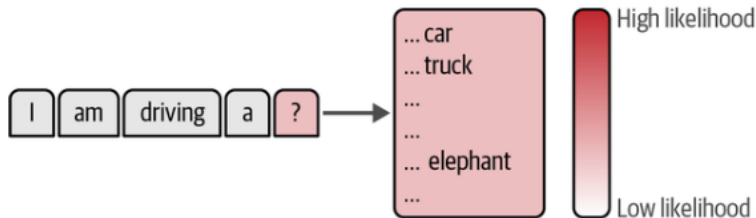


Department of Electrical Engineering and Computer Science
University of Stavanger

February 17, 2026



- ▶ LLM predicts next token
- ▶ Each token has probability score
- ▶ Sampling determines creativity



Temperature vs Top-p: What Do They Actually Do?

- ▶ Both modify token sampling from the model's probability distribution
- ▶ Temperature rescales logits before softmax
- ▶ Top-p truncates the candidate token set based on cumulative probability

Temperature (T)

$$P_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

- ▶ $T < 1$: sharper distribution $\rightarrow$ more deterministic
- ▶ $T > 1$: flatter distribution $\rightarrow$ more diverse

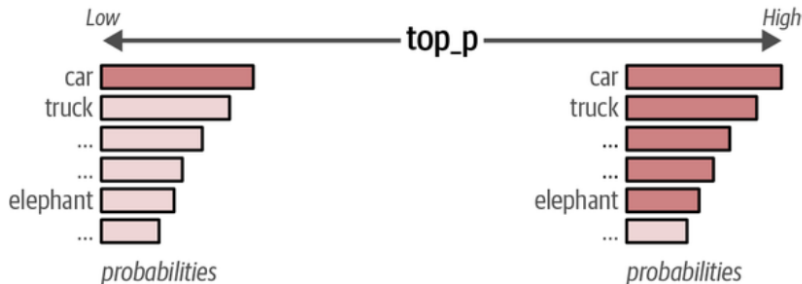


- ▶ Temperature T scales logits before softmax: $p_i \propto \exp(i/T)$.
- ▶ Low T (0–0.5) yields deterministic outputs, focusing on high-probability tokens.
- ▶ Moderate T (0.5–1.0) provides balanced diversity and coherence.
- ▶ High T (> 1) increases randomness, potentially generating creative but less reliable responses.
- ▶ $T = 0$ corresponds to greedy decoding without sampling.
- ▶ Choose T based on the desired trade-off between creativity and consistency.

Nucleus Sampling (Top-p)



- ▶ Sort tokens by probability (descending)
- ▶ Select smallest set where cumulative probability $\geq p$
- ▶ Sample only from this nucleus



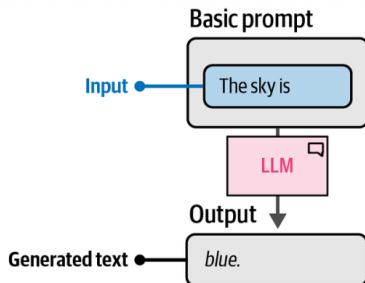


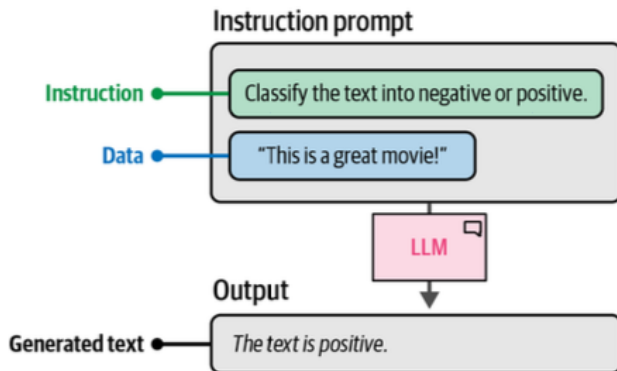
Use Case	Temperature	Top-p
Deterministic QA	0.0–0.3	0.1–0.5
Email writing	0.3–0.6	0.5–0.8
Creative writing	0.8–1.2	0.8–1.0
Brainstorming	1.0+	0.9–1.0

Key insight:

- ▶ Temperature changes *shape* of distribution
- ▶ Top-p changes *size* of candidate pool
- ▶ They interact but are not equivalent

- ▶ Designing inputs to guide LLM output
- ▶ Iterative optimization process
- ▶ No perfect prompt exists



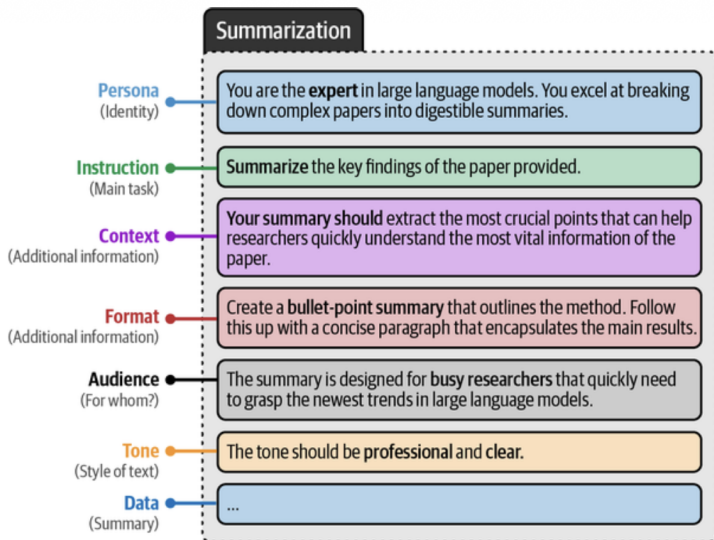




- ▶ Guide structured responses
- ▶ Example:

Text: I love this movie.
Sentiment:

Modular Prompt Components



Zero-shot, One-shot, Few-shot



- ▶ Zero-shot: no examples
- ▶ One-shot: single example
- ▶ Few-shot: multiple examples

Zero-shot prompt

Prompting without examples

Classify the text into neutral, negative, or positive.

Text: I think the food was okay.

Sentiment: ...

One-shot prompt

Prompting with a single example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**

Text: I think the food was okay.

Sentiment:

Few-shot prompt

Prompting with more than one example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**.

Text: I think the food was great!

Sentiment: **Positive**.

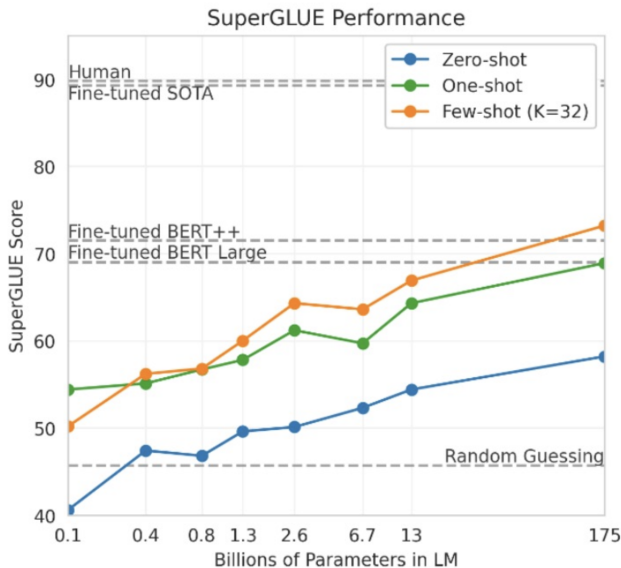
Text: I think the food was horrible...

Sentiment: **Negative**.

Text: I think the food was okay.

Sentiment:

Comparison of fine-tuned vs prompted



Comparison of fine-tuned vs prompted (machine translation)



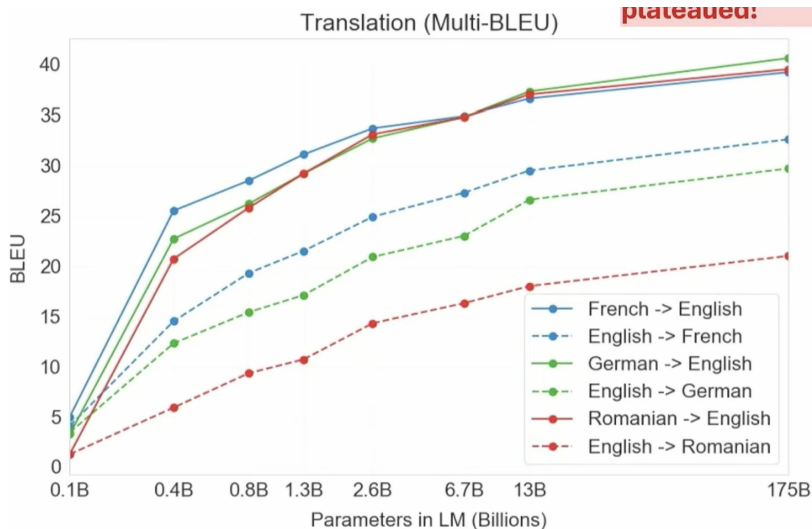
Setting	En→Fr	Fr→En	En→De	De→En	En→Ro	Ro→En
SOTA (Supervised)	45.6^a	35.0 ^b	41.2^c	40.2 ^d	38.5^e	39.9^e
XLM [LC19]	33.4	33.3	26.4	34.3	33.3	31.8
MASS [STQ ⁺ 19]	<u>37.5</u>	34.9	28.3	35.2	<u>35.2</u>	33.1
mBART [LGG ⁺ 20]	-	-	<u>29.8</u>	34.0	35.0	30.5
GPT-3 Zero-Shot	25.2	21.2	24.6	27.2	14.1	19.9
GPT-3 One-Shot	28.3	33.7	26.2	30.4	20.6	38.6
GPT-3 Few-Shot	32.6	<u>39.2</u>	29.7	<u>40.6</u>	21.0	<u>39.5</u>

Brown et al. 2020

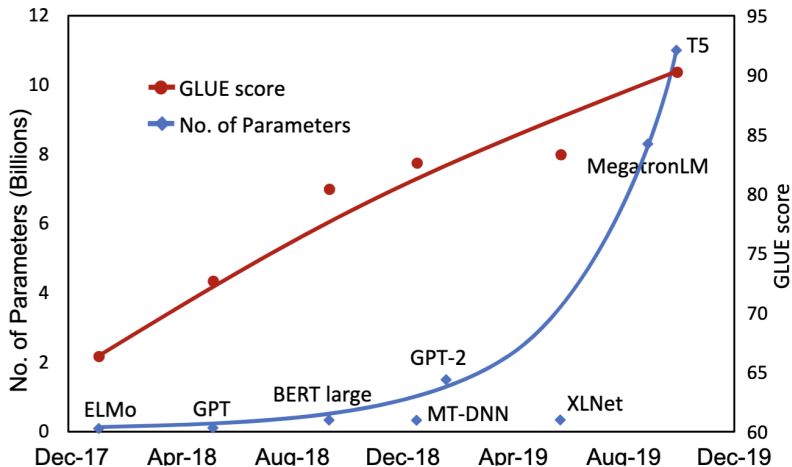
tion



Model size scaling



Model size limitation

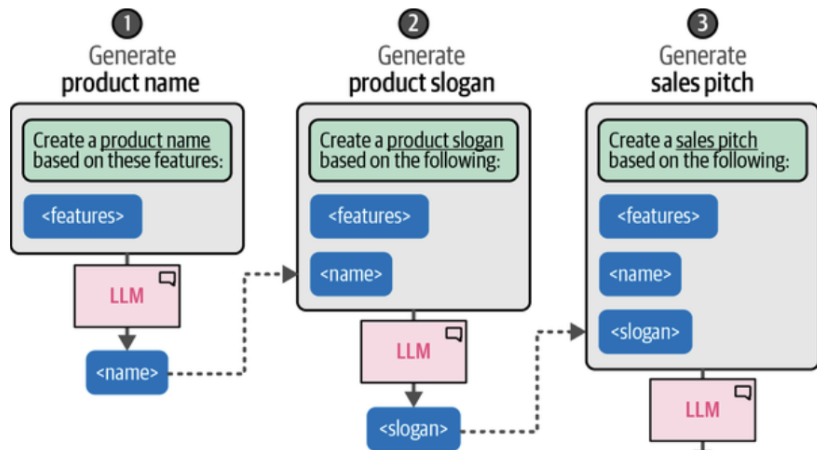


Ahmet et al. 2020

Chain Prompting



- Break complex tasks into steps
- Output of step i becomes input of step i+1
- Improves modularity and control





- ▶ “Think fast and slow” Kahneman 2011
- ▶ “System 1”: fast, intuitive (thinking represents an automatic, intuitive, and near-instantaneous process.)
- ▶ “System 2”: slow, deliberate (thinking is a conscious, slow, and logical process, akin to brainstorming and self-reflection.)
- ▶ Reasoning models mimic “system 2”

Chain-of-Thought

- ▶ Encourage step-by-step reasoning
- ▶ Improves multi-step tasks

One-shot prompt

Prompting with a single example

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The answer is 27. ❌

Chain-of-thought prompt

Prompting with a reasoning example

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls.
 $5 + 6 = 11$
The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$.
The answer is 9. ✅

Example

Reasoning process (thought)

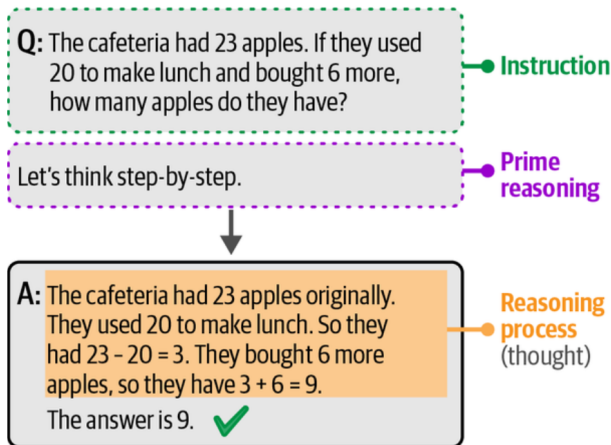
Instruction

Reasoning process (thought)

Zero-shot Chain-of-Thought

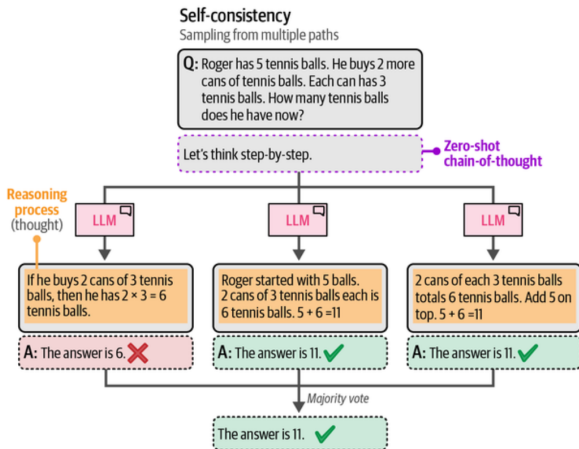


- ▶ Add phrase: “Let’s think step-by-step”
- ▶ No examples required





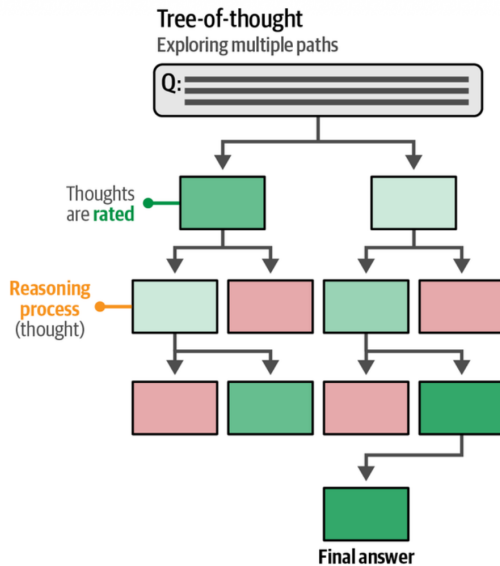
- ▶ Sample multiple reasoning paths
- ▶ Majority voting on final answer
- ▶ Improves robustness





- ▶ Explore multiple intermediate thoughts
- ▶ Score and prune branches
- ▶ Useful for planning and creativity

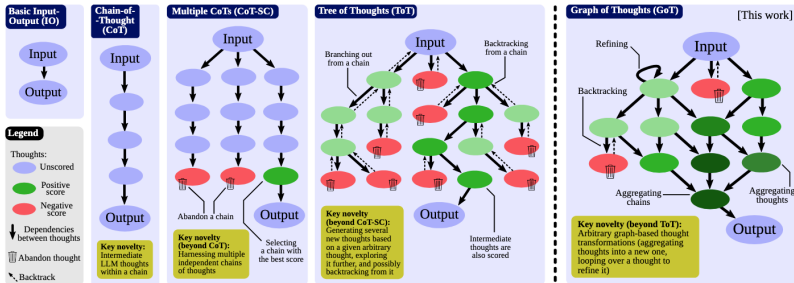
Tree-of-Thought (cont.)





- ▶ Explore multiple intermediate thoughts
- ▶ Score and prune branches
- ▶ Useful for planning and creativity

Graph-of-Thought (cont.)



Besta et al. 2024

Why Verify Output?



- ▶ Structured output (JSON)
- ▶ Valid labels
- ▶ Ethical constraints
- ▶ Accuracy



```
{  
  "description": "...",  
  "name": "...",  
  "weapon": "..."  
}
```



```
# One-shot learning: Providing an example of the output structure
one_shot_template = """Create a short character profile for an
RPG game. Make sure to only use this format:
```

```
{
  "description": "A SHORT DESCRIPTION",
  "name": "THE CHARACTER'S NAME",
  "armor": "ONE PIECE OF ARMOR",
  "weapon": "ONE OR MORE WEAPONS"
}
"""
one_shot_prompt = [
  {"role": "user", "content": one_shot_template}
]
```

```
# Generate the output
outputs = pipe(one_shot_prompt)
print(outputs[0]["generated_text"])
```

```
{
  "description": "A cunning rogue with a mysterious past,
skilled in stealth and deception.",
  "name": "Lysandra Shadowstep",
  "armor": "Leather Cloak of the Night",
  "weapon": "Dagger of Whispers, Throwing Knives"
}
```

- ▶ Constrain token selection
- ▶ Enforce JSON grammar
- ▶ Restrict allowed labels

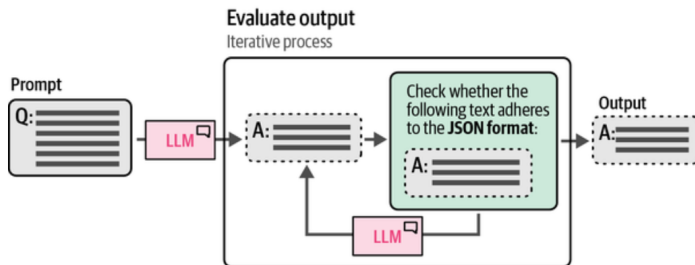
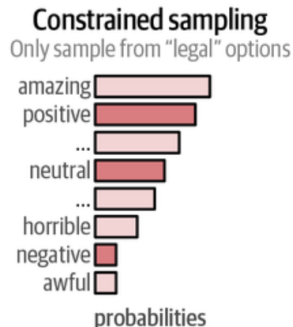
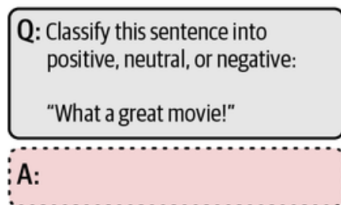


FIGURE 2.10 Grammar constrained sampling for JSON output





- Ahmet, A. et al. (2020). “Real-time social media analytics with deep transformer language models: A big data approach”. In: *2020 IEEE 14th International Conference on Big Data Science and Engineering (BigDataSE)*. IEEE, pp. 41–48.
- Alammar, J. et al. (2024). *Hands-On Large Language Models*. O'Reilly Media.
- Besta, M. et al. (2024). “Graph of thoughts: Solving elaborate problems with large language models”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 38. 16, pp. 17682–17690.
- Brown, T. et al. (2020). “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33, pp. 1877–1901.
- Kahneman, D. (2011). *Thinking, Fast and Slow*. Macmillan.